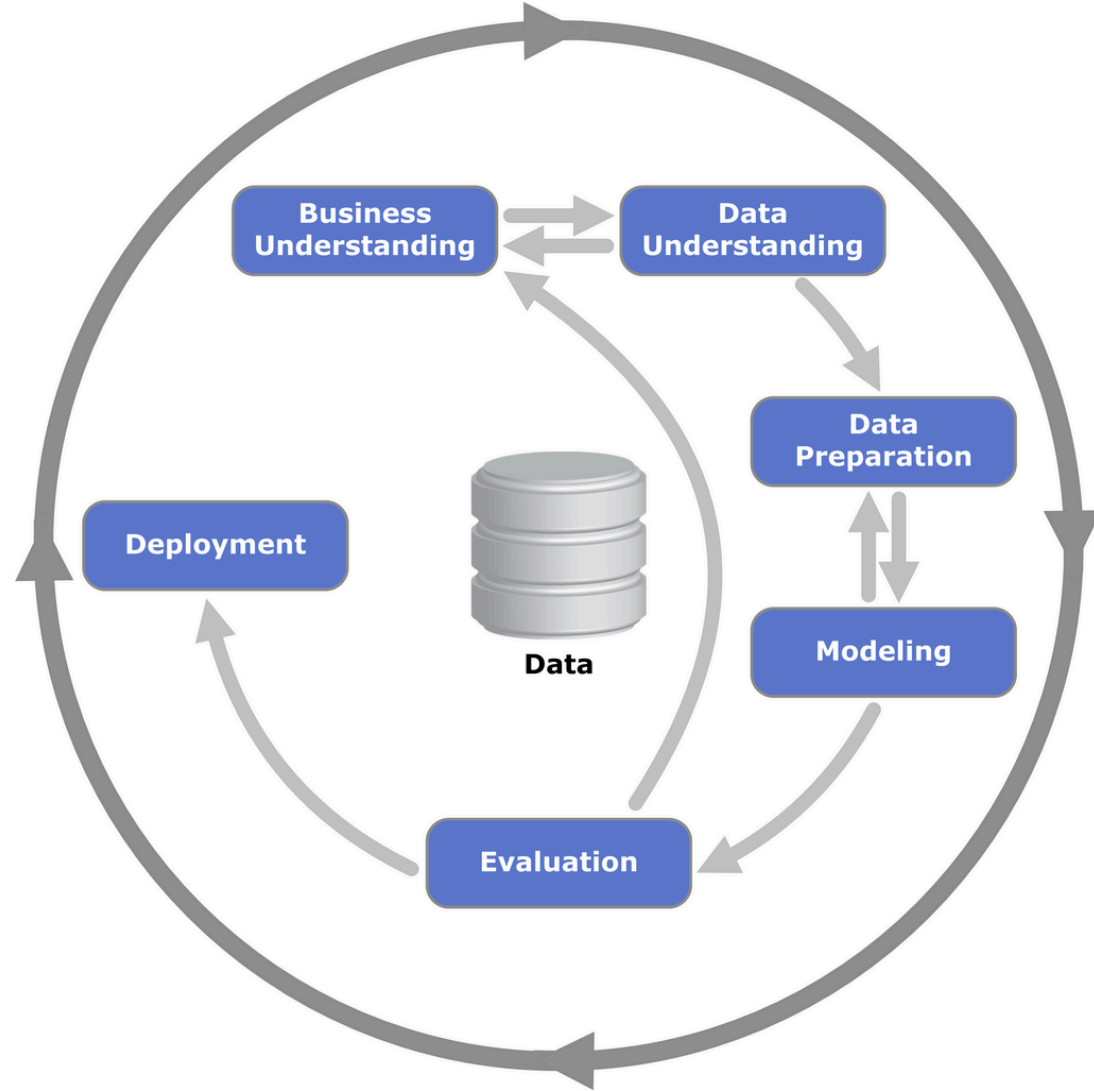




# Introducció al Machine Learning

- Predicció basada en dades (aprenentatge a partir de dades).
- Tipus:
  - Supervisat (*Supervised*):
    - Regressió (*Regression*)
    - Classificació (*Classification*)
  - No supervisat (*Unsupervised*)
  - Aprenentatge per reforç (*Reinforcement Learning*, RL)
- Aquí oferim una introducció al ML amb un problema supervisat.

# Dades

- Per aplicar ML necessitem dades (netes i processades).
- Botiga imaginària: dades sobre temperatura (°C) i vendes de gelats (€).
- Usem la temperatura (*feature*, característica) per predir les vendes (*target*).
- Les dades que usem per entrenar models s'anomenen *training data* (dades d'entrenament).

```
In [2]: df = pd.DataFrame({  
        'temp (x)': [5, 8, 10, 12, 20, 23, 33, 35],  
        'sales (y)': [200, 235, 210, 240, 420, 435, 560, 620]})  
df
```

```
Out[2]:
```

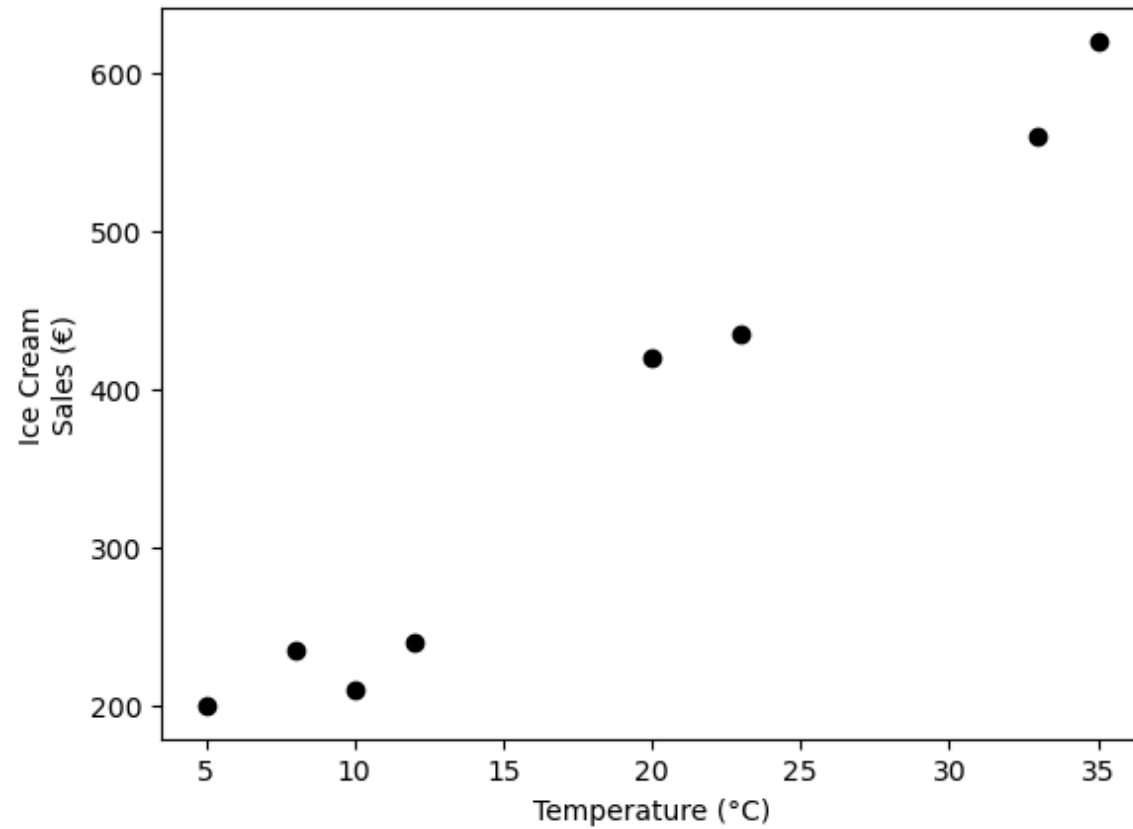
|   | temp (x) | sales (y) |
|---|----------|-----------|
| 0 | 5        | 200       |
| 1 | 8        | 235       |
| 2 | 10       | 210       |
| 3 | 12       | 240       |
| 4 | 20       | 420       |
| 5 | 23       | 435       |
| 6 | 33       | 560       |
| 7 | 35       | 620       |

- Els noms convencionals són:
  - `X` per referir-se a la matriu de **features** (característiques/atributs/variables) que ajuden a la predicció (`temperature` en aquest cas).
  - `y` per referir-se a l'atribut **target** (objectiu) que volem predir (`sales` en aquest cas).

```
In [3]: # temperature (°C)
X = df[['temp (x)']]

# sales (€)
y = df['sales (y)']
```

```
In [5]: plot_data(X, y)  
plt.show()
```



# Model Lineal

- Recta de la forma  $f(x) = w_0 + w_1 * x$ .
- P. ex.  $w_0 = 100$  i  $w_1 = 14$ .
- La nostra predicció de vendes de gelats és  $\hat{y} = 100 + 14 * x$ , on
  - $x$  és la temperatura.
- Usem  $\hat{y}$  per referir-nos a la predicció de la nostra aproximació de funció.
- Usem  $y$  per referir-nos als valors reals de vendes de les nostres dades.

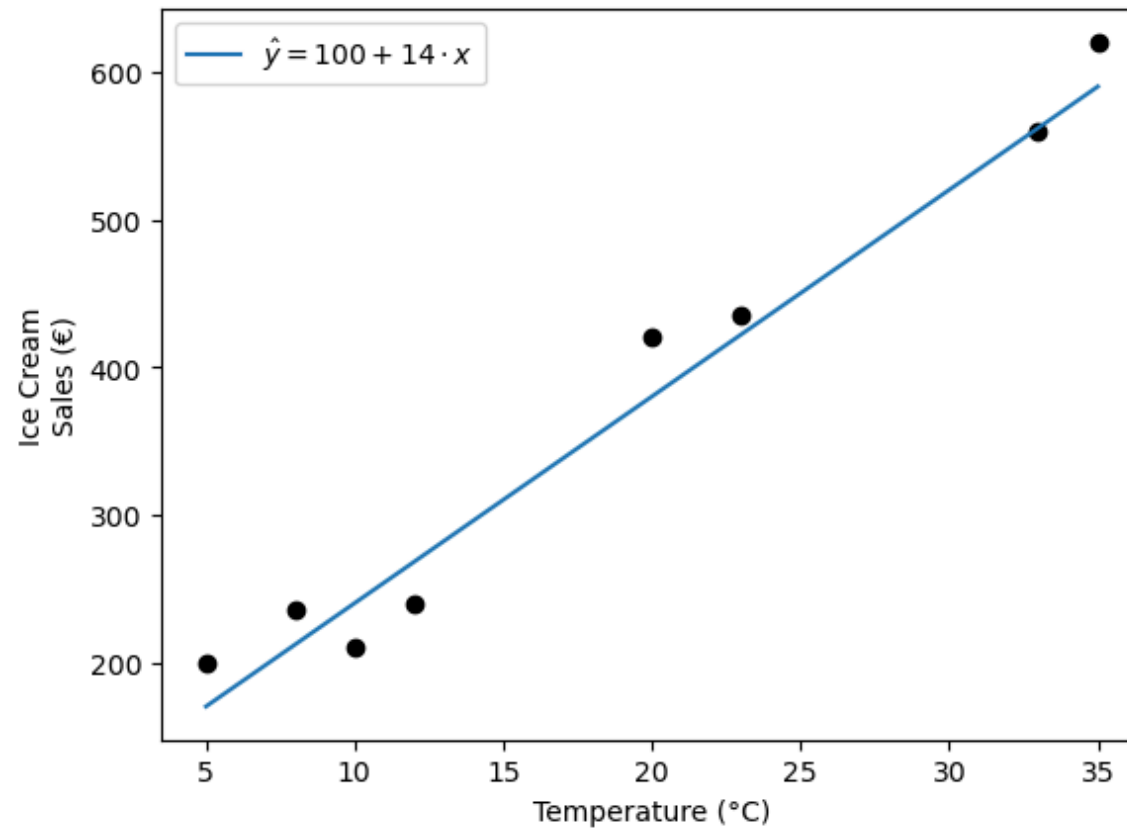
```
In [6]: # Model

w0 = 100
w1 = 14

# predicted y
y_pred = w0 + w1 * X
```

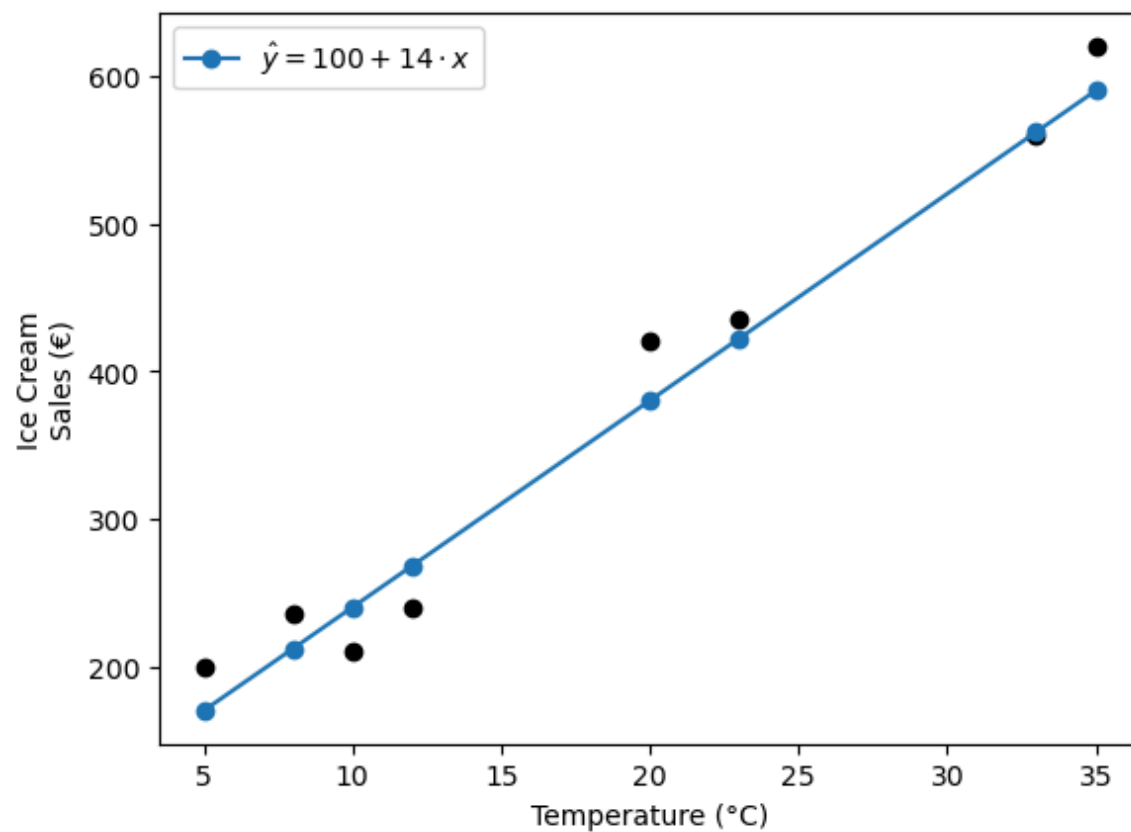
In [9]: *# Model*

```
plot_data(X, y)
plt.plot(X, y_pred, label=r"$\hat{y} = 100 + 14 \cdot x$")
plt.legend()
plt.show()
```



```
In [10]: # Predictions
```

```
plot_data(X, y)
plt.plot(X, y_pred, marker='o', label=r"$\hat{y} = 100 + 14 \cdot x$")
plt.legend()
plt.show()
```



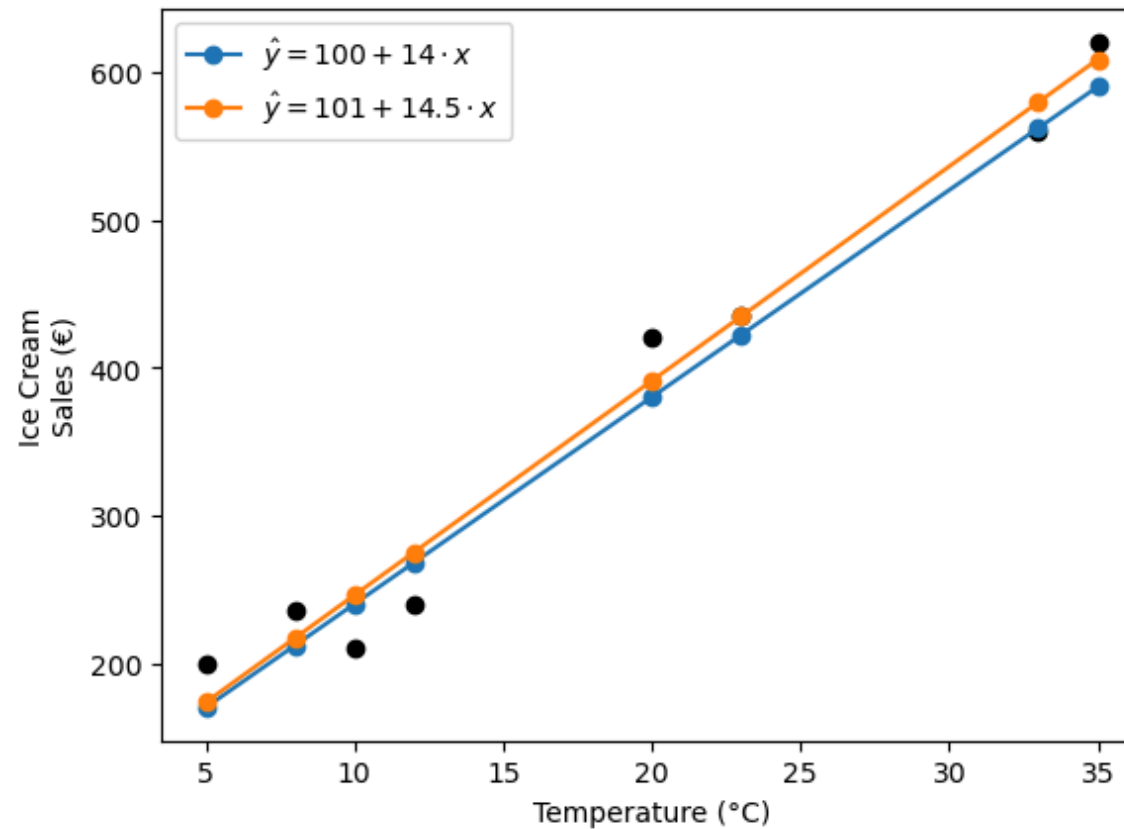
```
In [11]: df['sales_pred ( $\hat{y}$ )'] = y_pred
df
```

```
Out[11]:
```

|   | temp (x) | sales (y) | sales_pred ( $\hat{y}$ ) |
|---|----------|-----------|--------------------------|
| 0 | 5        | 200       | 170                      |
| 1 | 8        | 235       | 212                      |
| 2 | 10       | 210       | 240                      |
| 3 | 12       | 240       | 268                      |
| 4 | 20       | 420       | 380                      |
| 5 | 23       | 435       | 422                      |
| 6 | 33       | 560       | 562                      |
| 7 | 35       | 620       | 590                      |

- Les nostres prediccions són bones? Sembla un model raonable.
- Però és aquest altre model (taronja) millor o pitjor que l'anterior (blau)?
- Necessitem una manera de quantificar com de bones són les prediccions.

```
In [13]: # Predictions 2  
y_pred_model_2 = 101 + 14.5 * X  
plot_2_models(y_pred, y_pred_model_2)
```



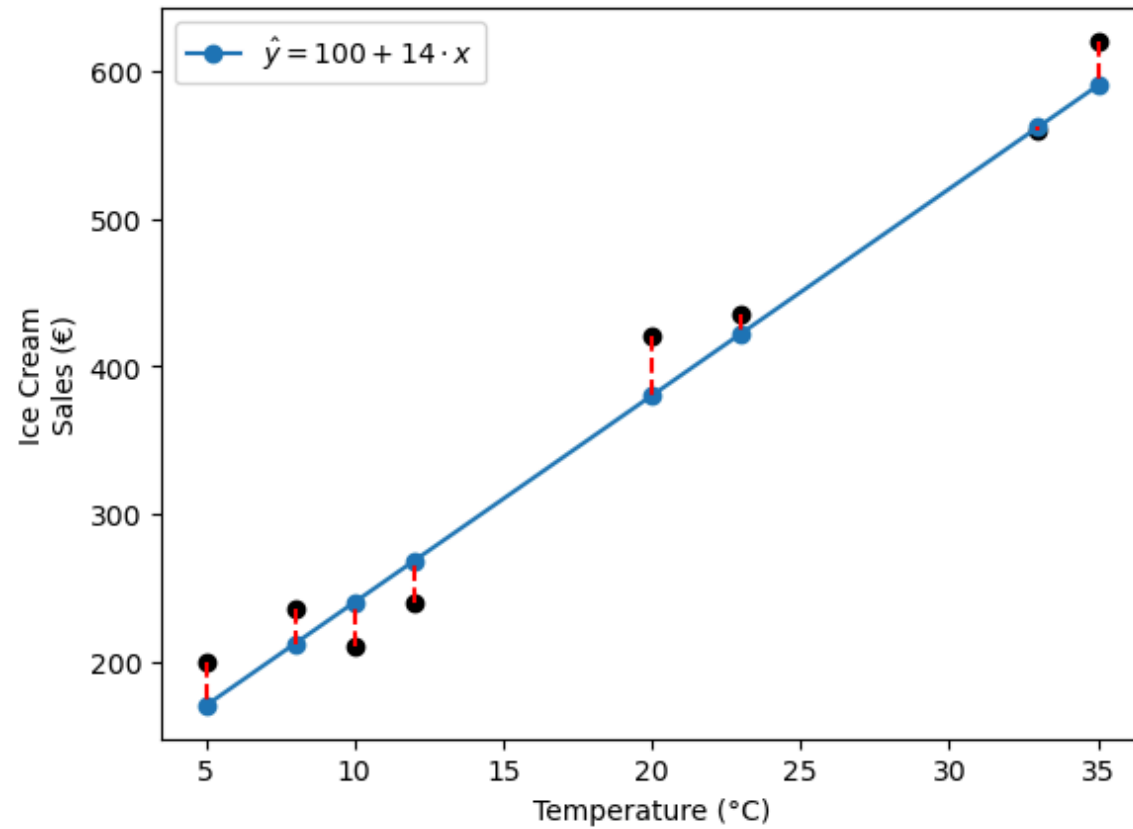
# Mètriques

- S'utilitzen per estimar l'error del nostre model.
- Tres mètriques típiques de regressió en ML (n'hi ha més):
  - Mean Absolute Error (MAE).
  - Mean Squared Error (MSE).
  - Root Mean Squared Error (RMSE).

## MAE

- Mesura la mitjana dels errors entre els valors predits i els observats (reals).
- $MAE = \frac{1}{N} \sum |y_i - \hat{y}_i|$ 
  - $y_i$  són els valors observats (dades reals).
  - $\hat{y}_i$  són els valors predits.
  - $N$  és el nombre d'observacions.

```
In [25]: # Predictions Errors  
plot_pred_errors()
```



```
In [26]: # Absolute error between each pair of observation-prediction  
df['abs_error (|y -  $\hat{y}$ |)'] = abs(df['sales (y)'] - df['sales_pred ( $\hat{y}$ )'])  
df
```

```
Out[26]:
```

|   | temp (x) | sales (y) | sales_pred ( $\hat{y}$ ) | abs_error ( y - $\hat{y}$  ) |
|---|----------|-----------|--------------------------|------------------------------|
| 0 | 5        | 200       | 170                      | 30                           |
| 1 | 8        | 235       | 212                      | 23                           |
| 2 | 10       | 210       | 240                      | 30                           |
| 3 | 12       | 240       | 268                      | 28                           |
| 4 | 20       | 420       | 380                      | 40                           |
| 5 | 23       | 435       | 422                      | 13                           |
| 6 | 33       | 560       | 562                      | 2                            |
| 7 | 35       | 620       | 590                      | 30                           |

```
In [27]: # MAE (sum of all elements divided by the length = mean)  
mae = df['abs_error (|y -  $\hat{y}$ |)'].mean()  
print(f'MAE: {mae} €')
```

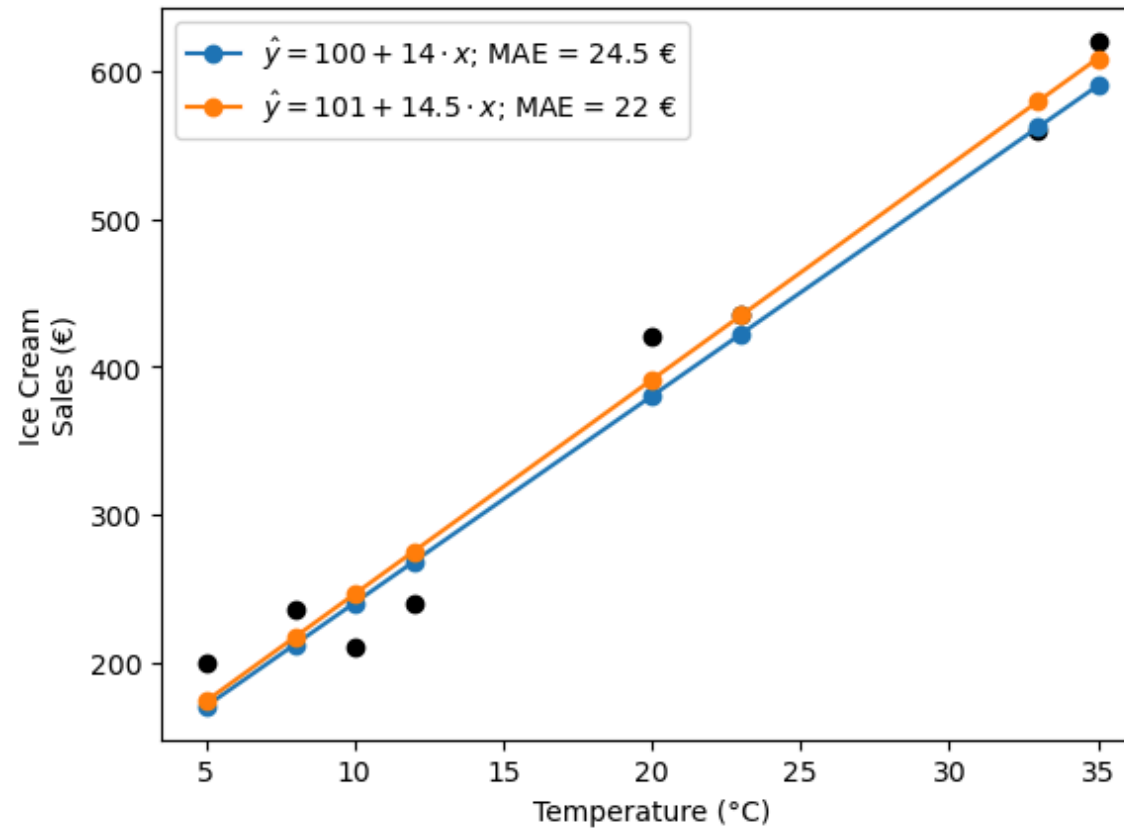
MAE: 24.5 €

Per simplicitat, podem usar una funció de `sklearn` per calcular el MAE:

```
In [32]: from sklearn.metrics import mean_absolute_error  
  
mae = mean_absolute_error(df['sales (y)'], df['sales_pred ( $\hat{y}$ )'])  
print(f'MAE: {mae} €')
```

MAE: 24.5 €

```
In [40]: plot_2_models_maes()
```



## MSE

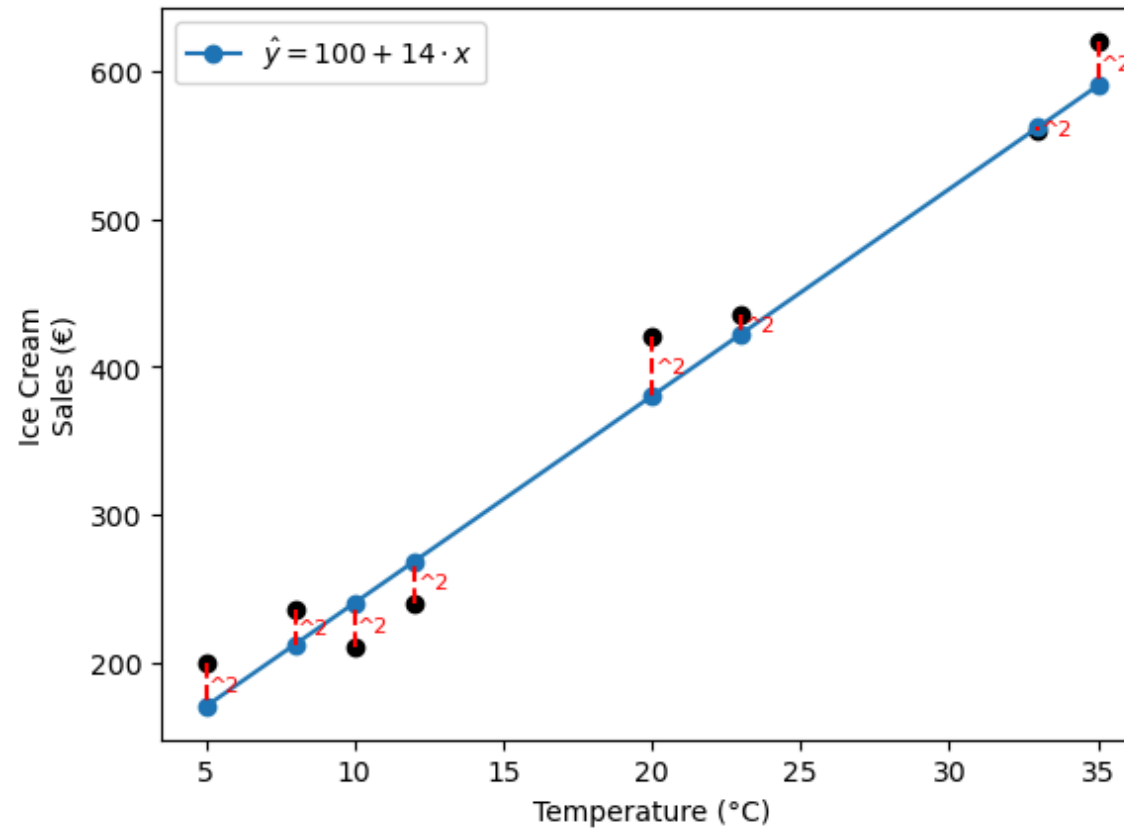
- Mesura la mitjana dels quadrats dels errors.
- $MSE = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$ 
  - $y_i$  són els valors observats (dades reals).
  - $\hat{y}_i$  són els valors predits.
  - $N$  és el nombre d'observacions.
- Més sensible als *outliers* (valors atípics) que el MAE.
- L'escala és al quadrat, costa d'interpretar... (€<sup>2</sup> en aquest cas).

## RMSE

- Mesura la mitjana dels quadrats dels errors.
- $RMSE = \sqrt{MSE}$
- Més sensible als *outliers* que el MAE.
- Està en l'escala de les unitats originals.

```
In [44]: # Predictions Error
plot_data(X, y)
plt.plot(X, y_pred, marker='o', label=r"$\hat{y} = 100 + 14 \cdot x$")

# Plotting vertical dashed lines
plot_dashed_squared(X, y, y_pred)
plt.legend()
plt.show()
```



```
In [56]: # Absolute error between each pair of observation-prediction  
sq_err = (df['sales (y)'] - df['sales_pred (ŷ)']) ** 2  
df['abs_err_sq ((y - ŷ)²)'] = sq_err  
df
```

```
Out[56]:
```

|          | <b>temp<br/>(x)</b> | <b>sales<br/>(y)</b> | <b>sales_pred<br/>(ŷ)</b> | <b>abs_error<br/>( y - ŷ )</b> | <b>root_abs_err_sq<br/>((y - ŷ)²)</b> | <b>abs_err_sq<br/>((y - ŷ)²)</b> |
|----------|---------------------|----------------------|---------------------------|--------------------------------|---------------------------------------|----------------------------------|
| <b>0</b> | 5                   | 200                  | 170                       | 30                             | 30.0                                  | 900                              |
| <b>1</b> | 8                   | 235                  | 212                       | 23                             | 23.0                                  | 529                              |
| <b>2</b> | 10                  | 210                  | 240                       | 30                             | 30.0                                  | 900                              |
| <b>3</b> | 12                  | 240                  | 268                       | 28                             | 28.0                                  | 784                              |
| <b>4</b> | 20                  | 420                  | 380                       | 40                             | 40.0                                  | 1600                             |
| <b>5</b> | 23                  | 435                  | 422                       | 13                             | 13.0                                  | 169                              |
| <b>6</b> | 33                  | 560                  | 562                       | 2                              | 2.0                                   | 4                                |
| <b>7</b> | 35                  | 620                  | 590                       | 30                             | 30.0                                  | 900                              |

```
In [57]: mse = df['abs_err_sq ((y -  $\hat{y}$ )2)'].mean()  
rmse = np.sqrt(mse)  
print(f'RMSE: {rmse:.1f} €2')
```

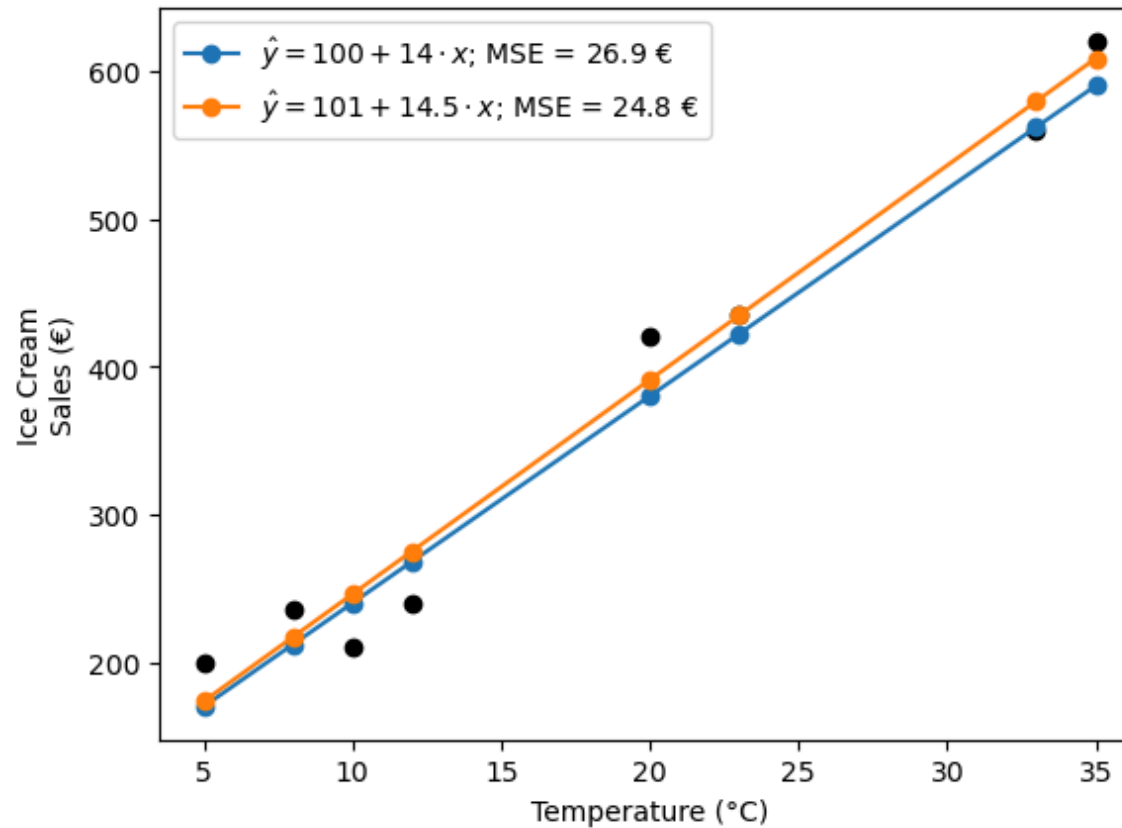
RMSE: 26.9 €<sup>2</sup>

Per simplicitat, podem usar una funció de `sklearn` per calcular el RMSE:

```
In [58]: from sklearn.metrics import root_mean_squared_error  
  
rmse = root_mean_squared_error(df['sales (y)'], df['sales_pred ( $\hat{y}$ )'])  
print(f'RMSE: {rmse:.1f} €')
```

RMSE: 26.9 €

```
In [62]: # Plot
plot_data(X, y)
lab = r"$\hat{y} = 100 + 14 \cdot x$; MSE = 26.9 €"
plt.plot(X, y_pred, marker='o', label=lab)
lab = r"$\hat{y} = 101 + 14.5 \cdot x$; MSE = 24.8 €"
plt.plot(X, y_pred_model_2, marker='o', label=lab)
plt.legend()
plt.show()
```

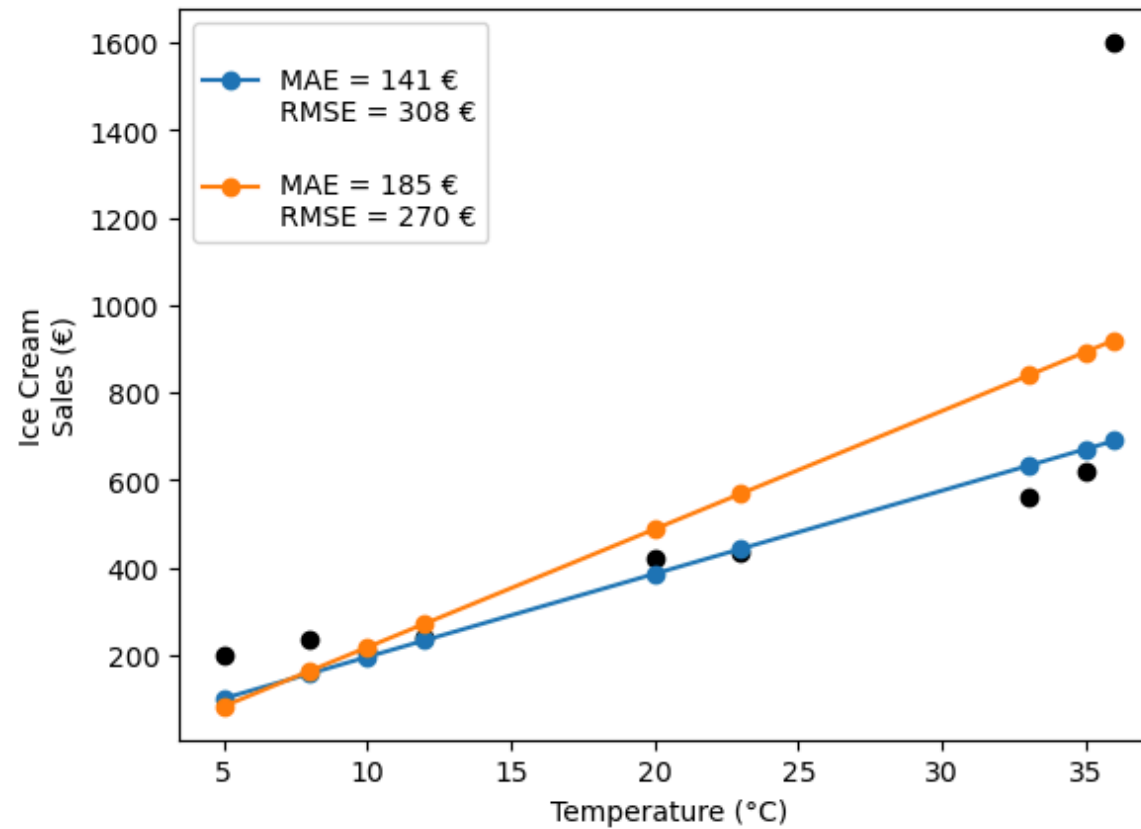


# MAE vs RMSE

- MAE:
  - Pren les diferències absolutes entre els valors reals i els predits.
  - Aplica una penalització lineal a cada error.
  - Menys sensible als *outliers* que el MSE i RMSE.
- RMSE:
  - Eleva al quadrat la diferència entre els valors reals i els predits.
  - Després "torna" a les unitats originals amb l'arrel quadrada.
  - Aplica una penalització quadràtica als errors.
  - Llavors en calcula l'arrel quadrada.
  - Més sensible als *outliers* que el MAE.

- Afegim un *outlier* (valor atípic) a les dades anteriors.
- El model blau és millor que el taronja amb MAE.
- El model taronja és millor que el blau amb RMSE.

In [72]: `plot_mae_vs_mse()`



## Quin és millor?

- Depèn.
- Sensibilitat del problema:
  - MAE si el problema no és sensible a errors grans.
  - RMSE si el problema és més sensible a errors grans.

# Machine Learning (ML)

- Després de:
  - Convertir les dades brutes (*raw*) en dades netes.
  - Processades i preparades per entrenar.
  - Seleccionar la(les) mètrica(ques) òptima(es) per a l'avaluació.
- Podem començar a aprendre (modelar) les dades.

# Linear Regression (Regressió Lineal)

- Ajusta una recta a les dades per fer prediccions,  $\hat{y} = w_0 + w_1 * x$ .
- Troba els coeficients òptims  $w_0$  i  $w_1$  basant-se en una mètrica.
- Optimitzat sobre MSE per defecte a `sklearn`.
- Considerant el nostre problema inicial (sense l'*outlier*):
  - Millors paràmetres:  $w_0 = 101.61$  i  $w_1 = 14.43$

```
In [73]: from sklearn.linear_model import LinearRegression
```

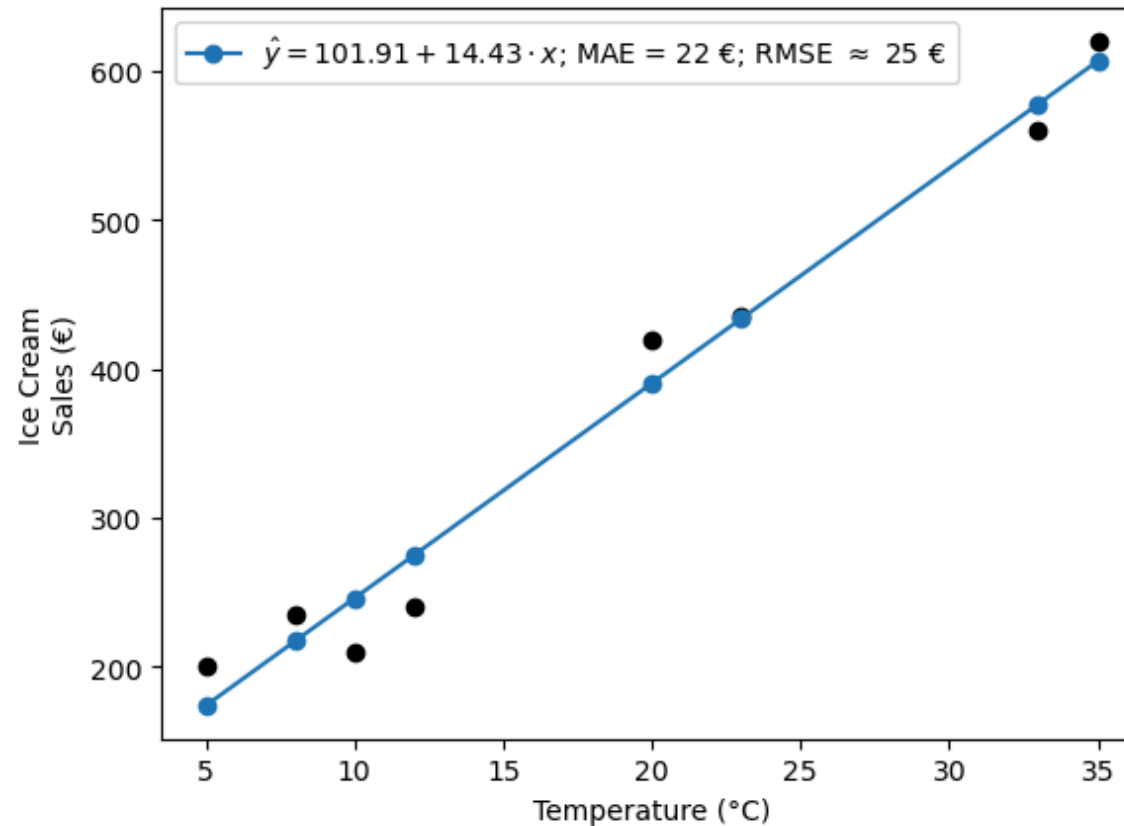
```
lr = LinearRegression().fit(X, y)
w0 = lr.intercept_
w1 = lr.coef_[0]
print('w0:', round(w0, 2))
print('w1:', round(w1, 2))

y_pred = w0 + w1 * X
```

```
w0: 101.61
w1: 14.43
```

In [77]: *# Predictions*

```
plot_data(X, y)
lab = r"$\hat{y} = 101.91 + 14.43 \cdot x$; MAE = 22 €; RMSE $\approx$ "
plt.plot(X, y_pred, marker='o', label=lab)
plt.legend()
plt.show()
```

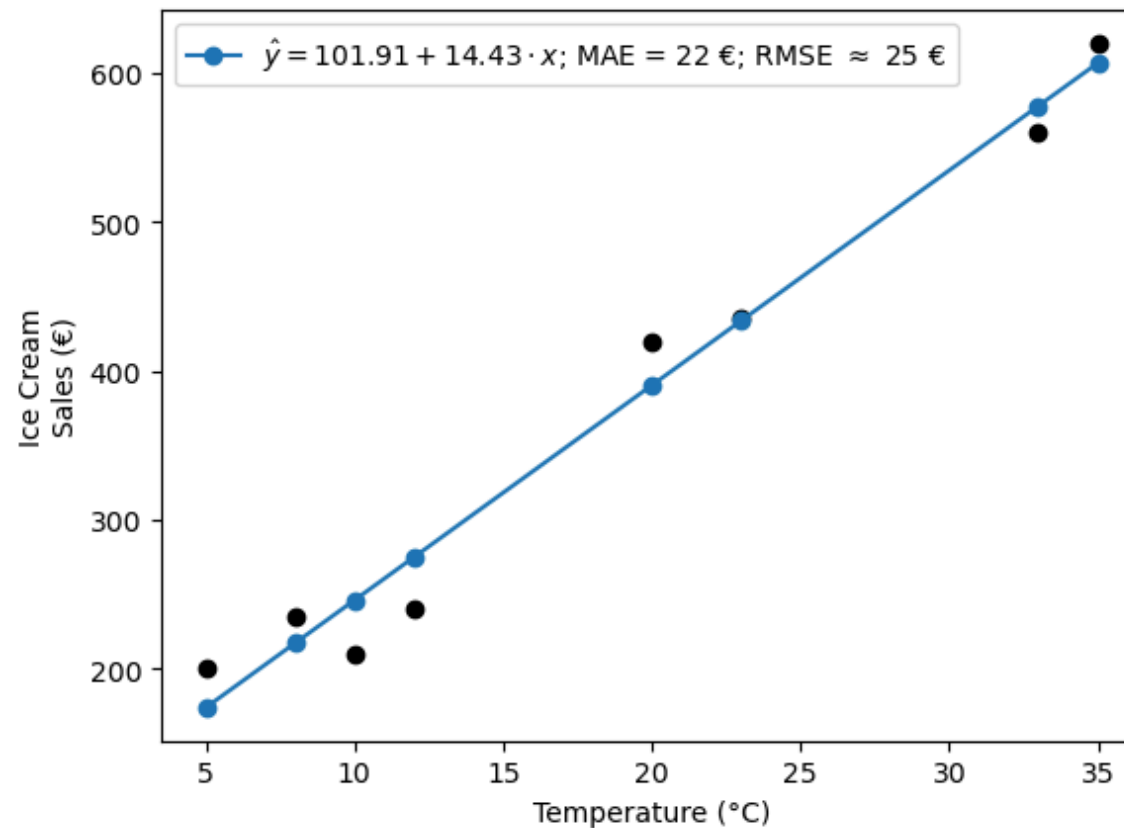


- No cal comprovar els valors dels paràmetres  $w_i$  cada vegada.
- Els models de `sklearn` tenen una funció `predict()` que prediu valors automàticament basant-se en un model entrenat.

```
In [78]: y_pred = lr.predict(X)
```

In [86]: *# Predictions*

```
plot_data(X, y)
lab = r"$\hat{y} = 101.91 + 14.43 \cdot x$; MAE = 22 €; RMSE $\approx$ "
plt.plot(X, y_pred, marker='o', label=lab)
plt.legend()
plt.show()
```



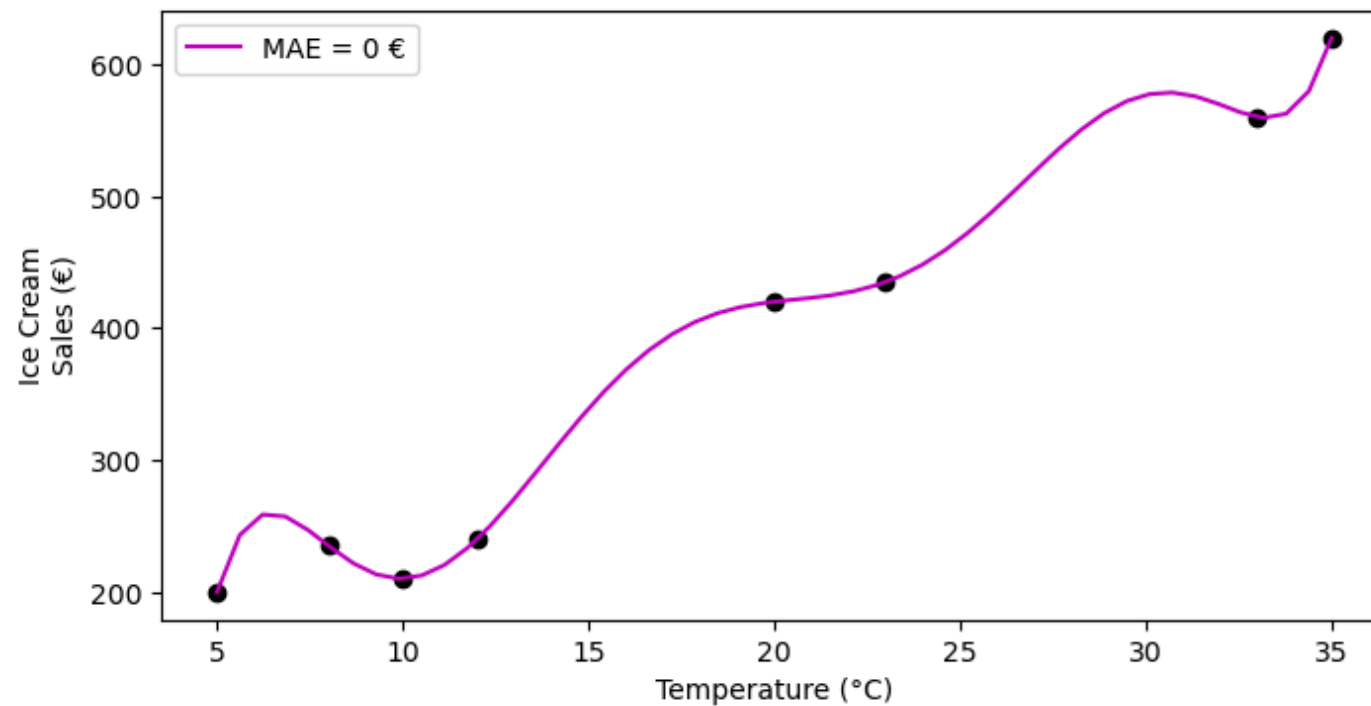
## Qualitat d'Ajust (*Quality of Fit*)

- Mesura com s'ajusten les prediccions del model amb les dades reals.
- Normalment s'avalua amb una o múltiples mètriques.
- Reptes clau:
  - *Overfitting* (sobreajust)
  - *Underfitting* (infraajust)

# Overfitting (Sobreajust)

- Fins i tot hi ha un model millor per a les dades!
- Mireu aquest, amb un  $MAE = 0$ , el model perfecte.
- Què en penseu?

In [92]: `plot_overfit()`



- Usem models per predir resultats futurs.
- Quan arriben dades noves, sabem si el nostre model les prediu bé o no.
- Podem usar la nova informació com a **test data** (dades de prova),
  - usant-les per avaluar el rendiment del nostre model sobre dades no vistes durant l'entrenament.
- Imagineu que les noves dades que rebem són:

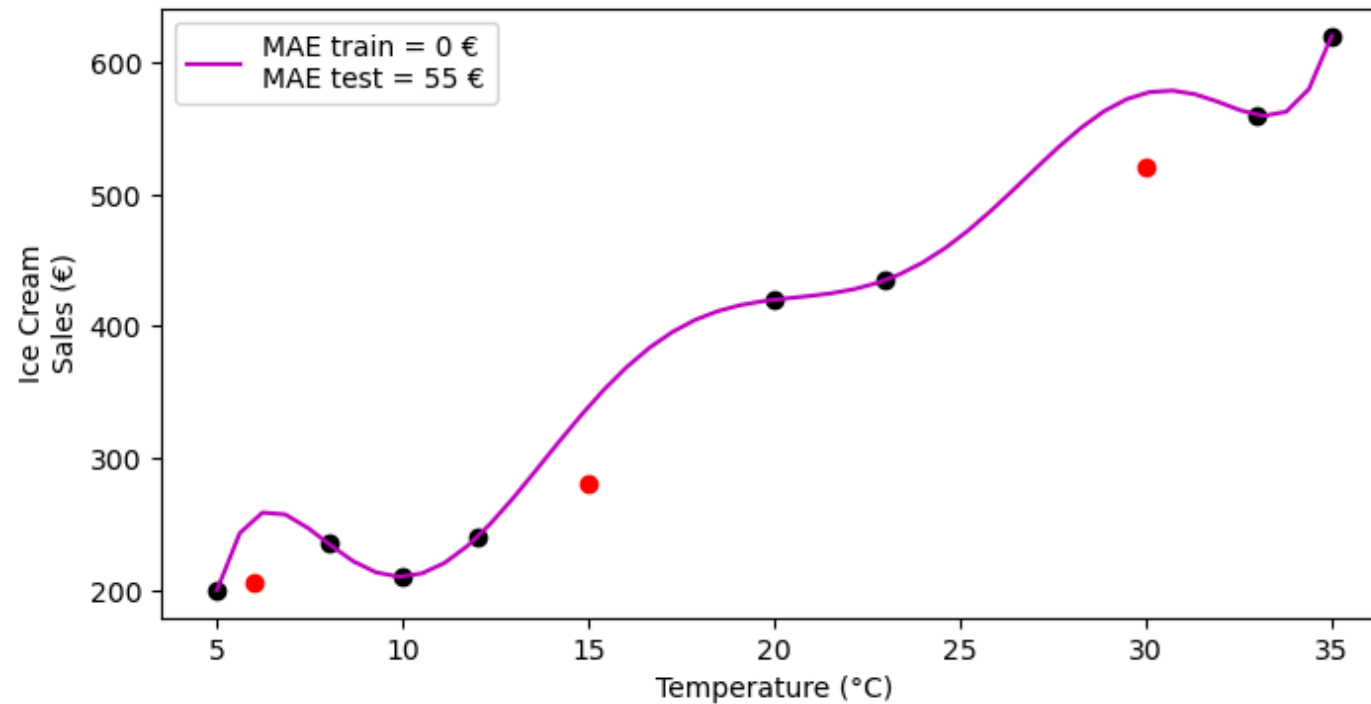
```
In [93]: df_test = pd.DataFrame({  
        'temp (x)': [6, 15, 30],  
        'sales (y)': [205, 280, 520]  
    })  
df_test
```

```
Out[93]:
```

|   | temp (x) | sales (y) |
|---|----------|-----------|
| 0 | 6        | 205       |
| 1 | 15       | 280       |
| 2 | 30       | 520       |

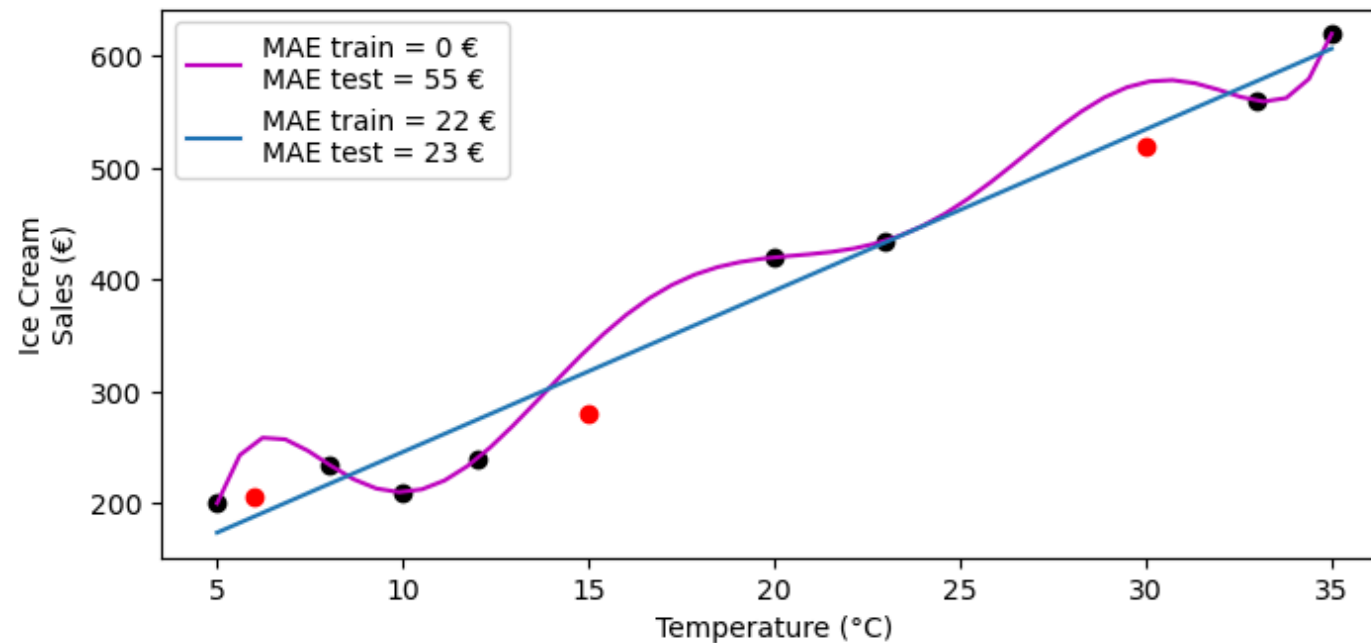
- Els punts negres són les dades d'entrenament (*train data*).
- Els punts vermells són les dades de prova (*test data*).
- Model polinomial:
  - Train MAE = 0 €
  - Test MAE = 55 €

In [97]: `plot_ovefit_test()`



- Model polinomial:
  - Train MAE = 0 €
  - Test MAE = 55 €
- El nostre model lineal anterior:
  - Train MAE = 22 €
  - Test MAE = 23 €

In [101]: `plot_overfit_n_linear()`

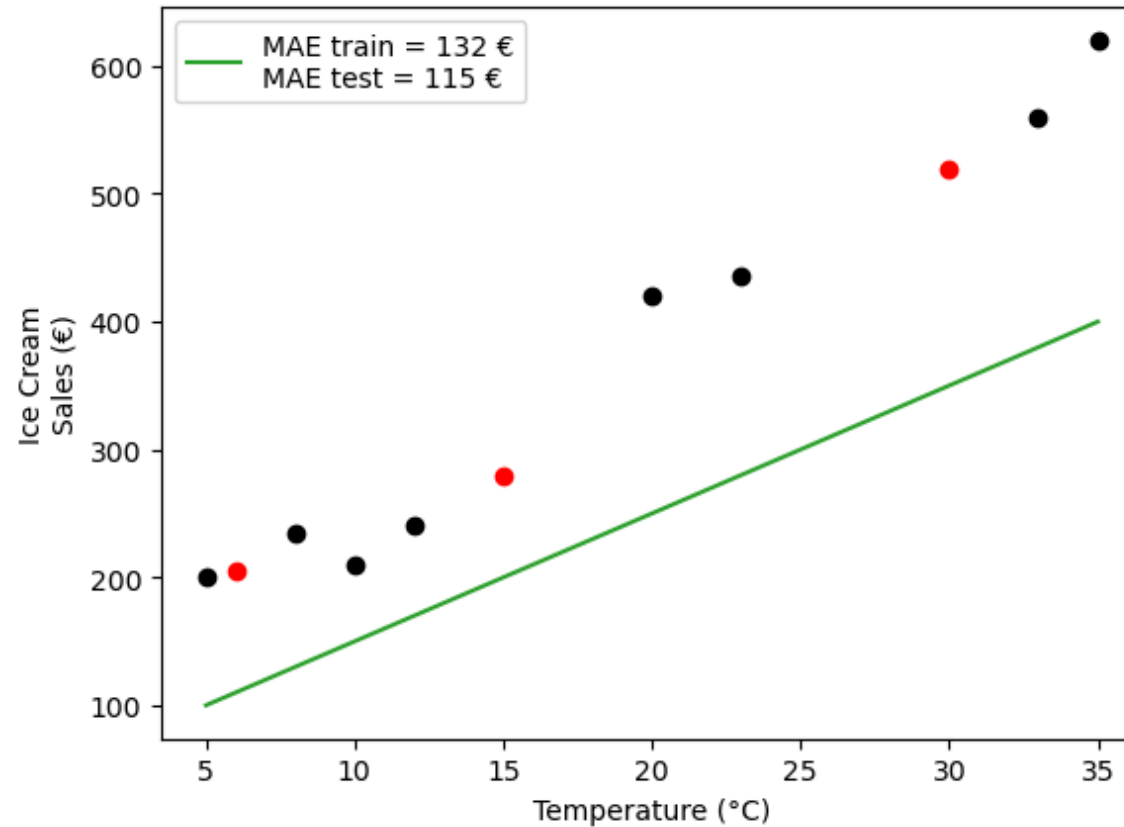


- Aquest efecte s'anomena ***overfitting***:
  - El model polinomial s'ha adaptat massa a les dades de train.
  - El rendiment és molt millor en train que en test.
- El model lineal té un rendiment similar en train i en test.
  - L'error no és massa alt.
  - Això pot indicar que el nostre model dona bones estimacions.

## Underfitting (Infraajust)

- El model té un rendiment dolent tant en les dades de train com en les de test.
- El model no captura els patrons subjacents de les dades.

```
In [103]: plot_underfit()
```

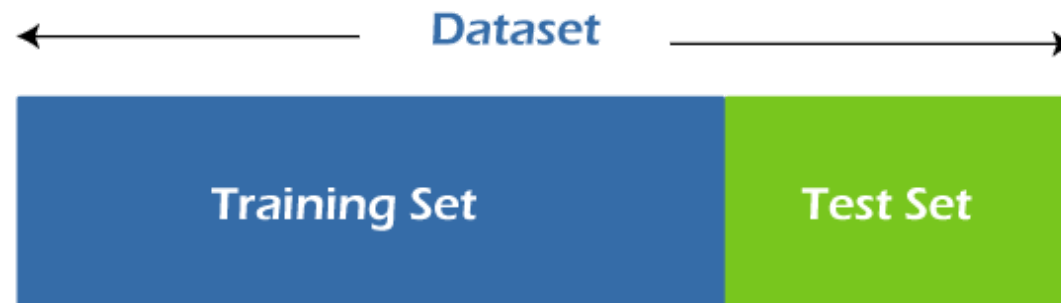


# Confiança en les Mètriques

- Visualitzar el model és factible en 2 o 3 dimensions, però es torna impossible amb desenes, centenars, milers o milions de dimensions.
- En canvi, confiem en les mètriques per avaluar el rendiment del model.
- Interpretació de les mètriques:
  - **Bon model:** L'error de train i de test és baix.
  - **Overfitting** (possible): Bon rendiment de train però baix de test.
    - També podria ser que les dades de test difereixin de les de train.
  - **Underfitting** (possible): Tant els errors de train com els de test són dolents.

# Train / Test

- Esperar dades noves per avaluar el nostre model seria una pèrdua de temps.
- Podem emular dades no vistes dividint les nostres dades en conjunts de **train** (entrenament) i **test** (prova).
  - Usar el conjunt de train per ajustar el nostre model.
  - Usar el conjunt de test per avaluar el rendiment del nostre model sobre dades no vistes durant l'entrenament.
- Les mides habituals del test són del 10-30% de les dades inicials.
  - *Datasets* grans poden requerir menys percentatge (p. ex., 10-20%).
  - *Datasets* petits poden requerir un conjunt de test més gran per assegurar que sigui representatiu (p. ex., 30-40%).
  - Durant la *cross-validation* (ho veurem més endavant) els conjunts de test poden ser més petits.



ML senzill:

1. Usar `train_test_split` de `sklearn` per dividir les dades:
  - **X\_train**: features usades per entrenar el model.
  - **X\_test**: features usades per provar el rendiment del model.
  - **y\_train**: target per al train.
  - **y\_test**: target per al test.
2. Ajustar un model al conjunt de train.
3. Comprovar el rendiment del model en els conjunts de train i test.

```
In [104]: from sklearn.model_selection import train_test_split
```

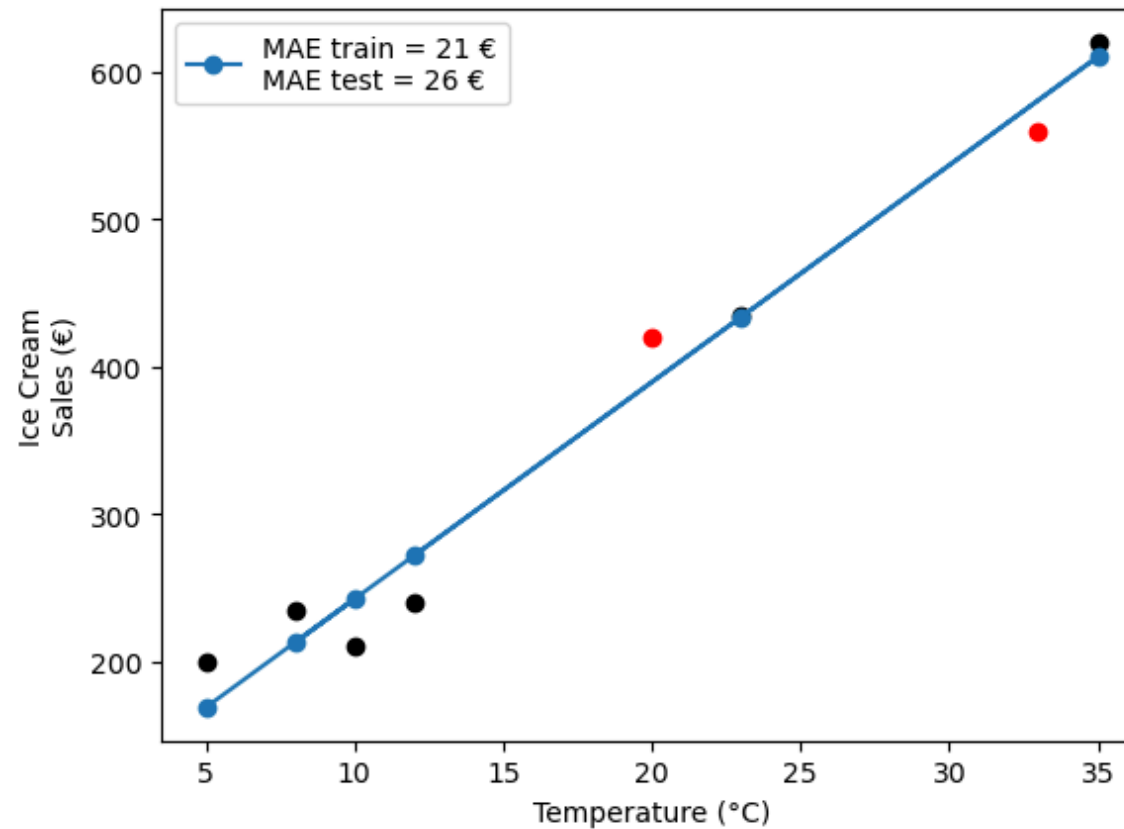
```
In [105]: # Using 20% of the data for testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    shuffle=True,
    random_state=12
)
```

```
In [106]: # Fit LR to train data (search for the optimal line based on MSE)  
lr = LinearRegression()  
lr.fit(X_train, y_train)  
  
# Predicted values on train and test sets  
y_pred_train = lr.predict(X_train)  
y_pred_test = lr.predict(X_test)
```

```
In [107]: # Compute train and test MAEs  
  
mae_train = mean_absolute_error(y_train, y_pred_train)  
mae_test = mean_absolute_error(y_test, y_pred_test)
```

```
In [110]: # Predictions

plot_data(X_train, y_train)
plot_data(X_test, y_test, color='r')
lab = f"MAE train = {mae_train} €\nMAE test = {mae_test} €"
plt.plot(X_train, y_pred_train, marker='o', label=lab)
plt.legend()
plt.show()
```



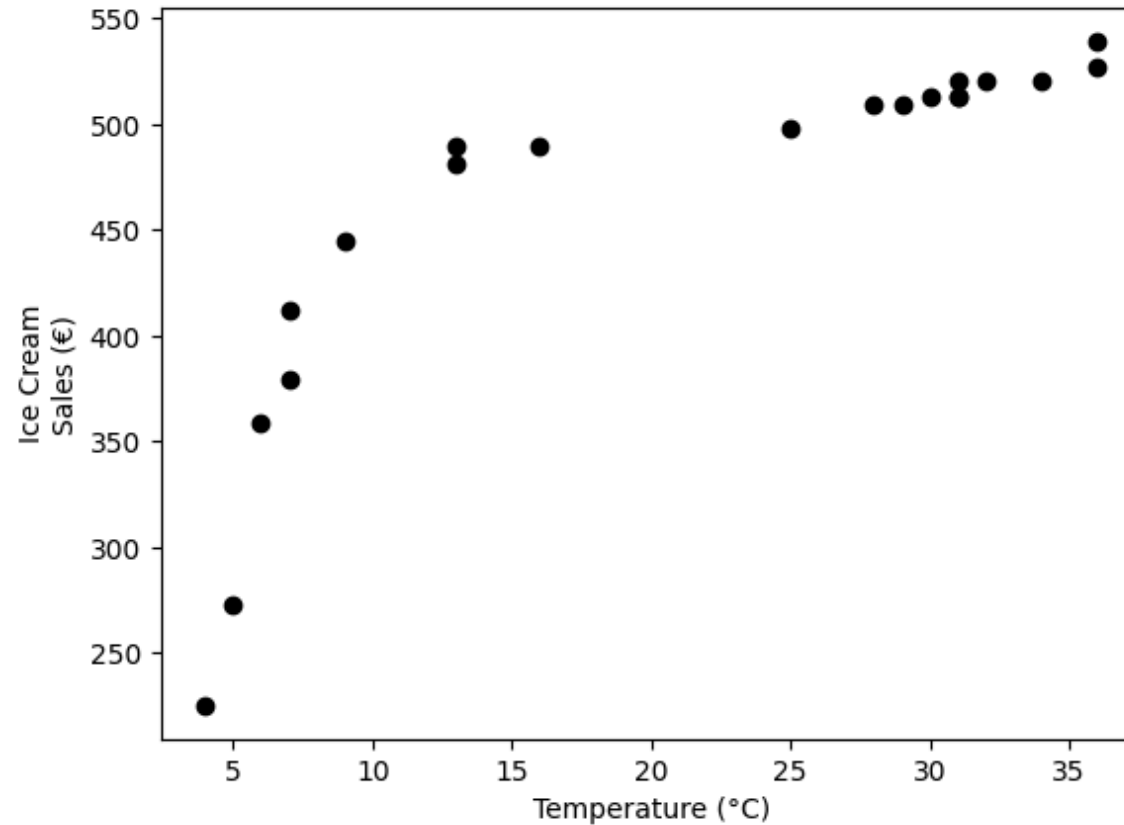
# Algorismes de ML

- Hi ha molts algorismes de ML més enllà de la Linear Regression (LR).
- Cadascun amb els seus avantatges i inconvenients.
- Alguns exemples:
  - Lasso
  - Ridge
  - ElasticNet
  - k-Nearest Neighbors (kNN)
  - Decision Tree (DT)
  - Random Forest (RF)
  - Gradient Boosted Decision Trees (GBDT)
  - Support Vector Machine (SVM)
  - Neural Network (NN)
  - ...

# Dades No Lineals

- Suposem que les nostres dades no són lineals.

```
In [106]: plot_nonlinear()
```



In [ ]: *# Using 20% of the data for testing*

```
X_train, X_test, y_train, y_test = train_test_split(  
    X.reshape(-1, 1),  
    y,  
    shuffle=True,  
    test_size=0.2,  
    random_state=12  
)
```

## Linear Regression

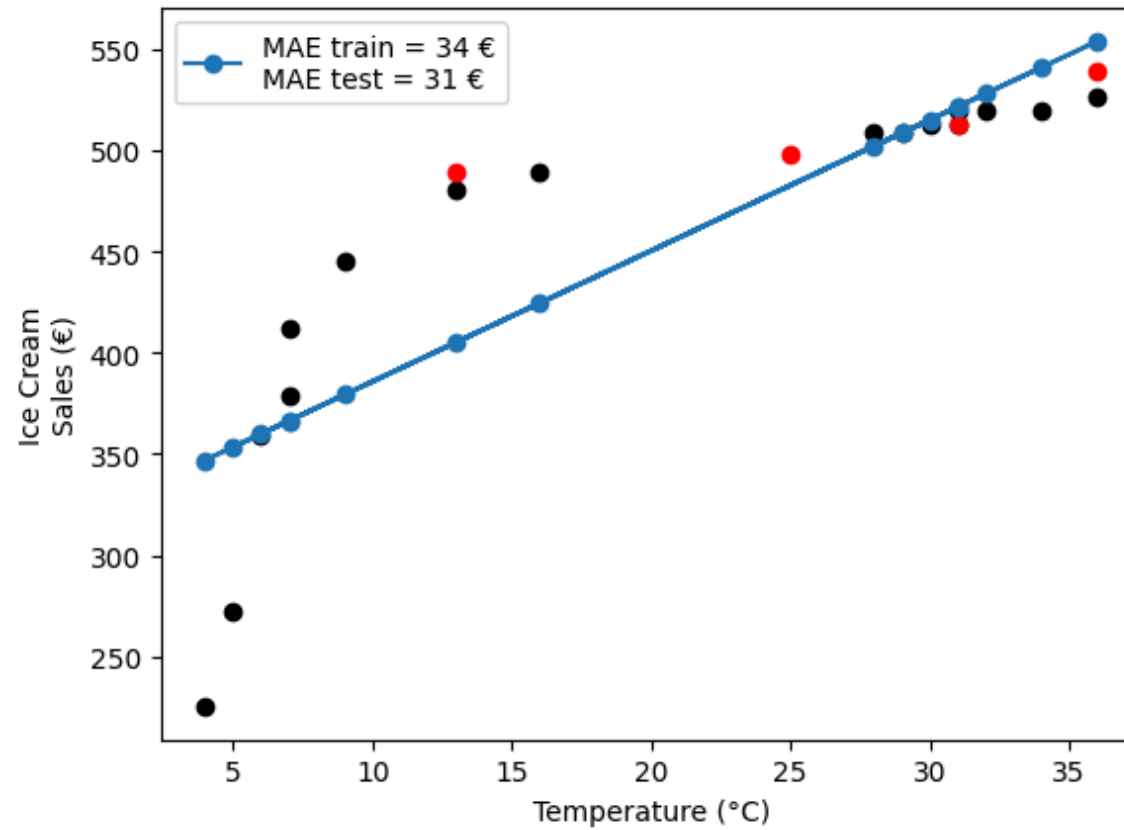
- Difícil fer bones prediccions ajustant una recta a dades no lineals.

```
In [108]: # Train
lr = LinearRegression()
lr.fit(X_train, y_train)

# Predict
y_pred_train = lr.predict(X_train)
y_pred_test = lr.predict(X_test)

# Error
mae_train = mean_absolute_error(y_train, y_pred_train)
mae_test = mean_absolute_error(y_test, y_pred_test)
```

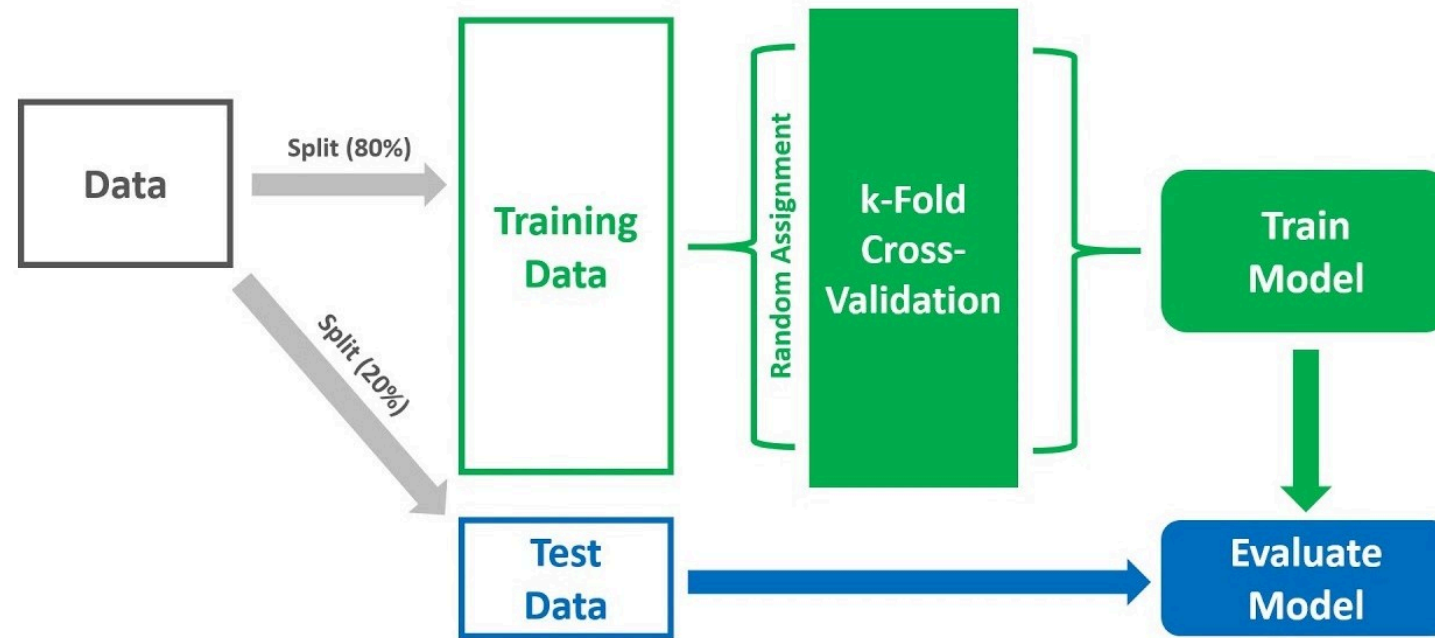
```
In [110]: plot_nonlinear_lr()
```



# Divisió Única Train / Test

- Problemes principals:
  - La divisió pot no representar la distribució global de les dades, provocant avaluacions enganyoses.
  - *Overfitting* sobre les dades de test: un bon rendiment en el conjunt de test pot no generalitzar-se a altres dades.
- En el futur parlarem d'una tècnica per superar alguns d'aquests problemes (s'anomena *Cross-Validation*).
- O una tècnica encara més robusta seria el *Nested Cross-validation* (no el veurem en aquest curs).

# Full ML Flow



## Més d'1 Feature

- Fins ara, visualitzar-ho tot amb 1 feature era fàcil.
- A la vida real, rarament tenim 1 sola feature.
- Podríem tenir fins a milions de features.

## 2 Features

- Afegim una altra feature a les nostres dades inicials: `wind` (en km/h).
- Ara amb `temperature` i `wind` com a features volem predir `sales`.
- Encara és fàcil de visualitzar en 3D.

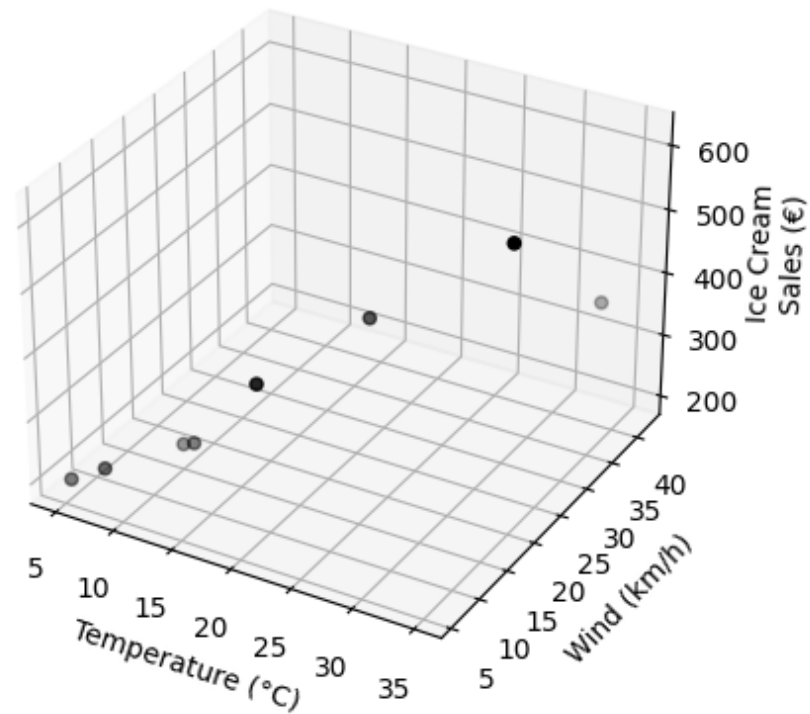
In [135]:

```
df2
```

Out[135]:

|          | <b>temp (x1)</b> | <b>wind (x2)</b> | <b>sales (y)</b> |
|----------|------------------|------------------|------------------|
| <b>0</b> | 5                | 6                | 200              |
| <b>1</b> | 8                | 6                | 235              |
| <b>2</b> | 10               | 15               | 210              |
| <b>3</b> | 12               | 13               | 240              |
| <b>4</b> | 20               | 8                | 420              |
| <b>5</b> | 23               | 21               | 435              |
| <b>6</b> | 33               | 42               | 350              |
| <b>7</b> | 35               | 20               | 620              |

```
In [137]: plot_3d_data(df2['temp (x1)'], df2['wind (x2)'], df2['sales (y)'])  
plt.show()
```

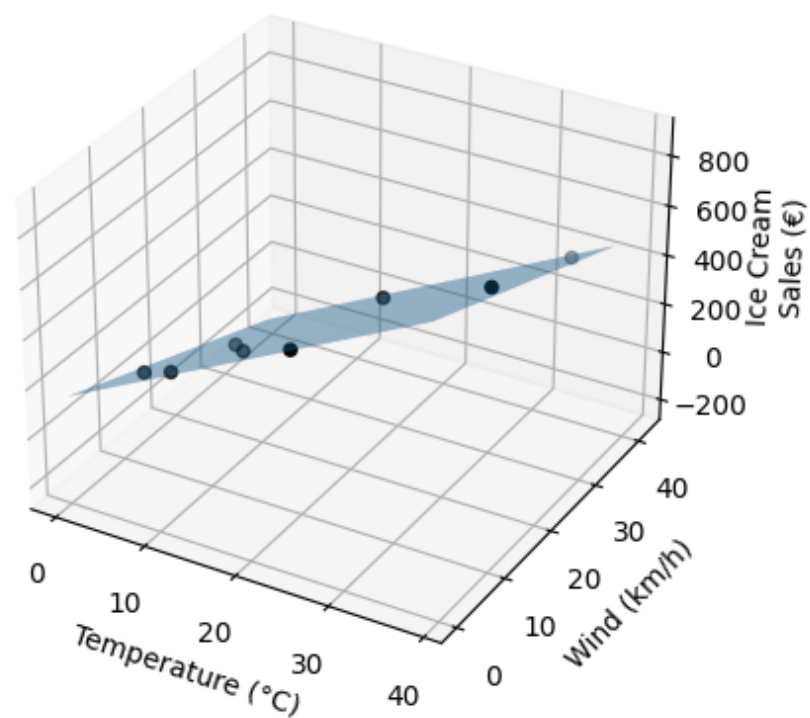


- També podem visualitzar els models donats per LR o kNN.
- Per a LR, tenim un pla en lloc d'una recta, amb una equació de la forma:
  - $\hat{y} = w_0 + w_1 * x_1 + w_2 * x_2$ , on
    - $x_1$  és la temperatura.
    - $x_2$  és la velocitat del vent.
  - Ara la Linear Regression troba els paràmetres òptims  $w_0$ ,  $w_1$  i  $w_2$ .

```
In [139]: plot_3d_plane()
```

Equation:  $\hat{y} = 155.12 + 18.38 * x_1 + -9.27 * x_2$

Linear Regression



## Múltiples Features

- Amb més de 2 features no és possible visualitzar-ho directament.
- Hi ha algunes tècniques, però no formen part d'aquest curs.
- Hem de confiar en els valors que ens donen les mètriques.

## Dummy / Línia de Base

- Tenim dades multidimensionals ( $> 3$  columnes).
- No podem visualitzar el model  $\rightarrow$  ens hem de fiar de les mètriques.
- Apliquem LR  $\rightarrow$  MAE train = 24€, MAE test = 26€.
- **És això bo o dolent?**

## Què és un Model Dummy?

- **Model naïf** que serveix com a **punt de referència** (*baseline*).
  - No aprèn de les dades: usa una regla molt simple.
  - Tipus comuns:
    - **Regressió**: prediu sempre la **mitjana** del *target*.
    - **Classificació**: prediu sempre la **classe més freqüent**.
  - Si el nostre model **no supera el dummy** → no millora una predicció trivial.
- 

- Exemple: volem predir vendes de gelats d'avui.
  - **Dummy** (prediu la mitjana) → MAE = 80€.
  - **LR** → MAE = 26€ ✓ La LR supera clarament el dummy.
- Si el dummy donés MAE = 20€ → la LR (26€) no estaria fent res massa útil.

# Resum

1. Netejar i preprocessar les dades.
2. Decidir quines **mètriques** s'adaptin millor al problema, p. ex., **MAE** o **RMSE**.
3. Dividir les dades en conjunts de **train** i **test**.
4. Provar resultats amb el model *dummy*.
5. Decidir quins algorismes usar per modelar les dades d'entrenament, p. ex., **LR** o qualsevol altre.
6. Anar amb compte amb l'***overfitting*** i l'***underfitting***.
7. Comprovar el rendiment d'aquests models en el conjunt de test.
  - Anar amb compte amb *overfitting* i *underfitting*.
  - No comprovar el rendiment en el conjunt de test constantment, o hi haurà *overfitting* sobre ell.
8. Conservar el millor model o tornar a un pas anterior si el rendiment no és bo.